AMRITA VISHWA VIDYAPEETHAM | School of Engineering Chennai Campus

Department of Computer Science and Engineering
2024 CSE BATCH
23CSE463 – Quantum Computing

KANTAMRAJU SREEJA

CH.SC.U4CSE24255

WORKSHEET-2

WORKSHEET- 2

EX.NO : 2

DATE : 09-10-26

TITLE : Implementation of Quantum Algorithms (Deutsch, Deutsch-Jozsa, Bernstein-Vazirani and Simon) Using Qiskit

AIM:

To implement the Deutsch, Deutsch-Jozsa, Bernstein-Vazirani and Simon algorithms in Qiskit, to construct the oracle for each, and to compare the number of oracle queries with the number a classical method needs.

APPARATUS REQUIRED:

- Python 3.10 or above
- Qiskit 2.x and Qiskit Aer simulator (`pip install qiskit qiskit-aer`)
- Matplotlib and pylatexenc (for circuit diagrams and histograms)
- Jupyter Notebook / Visual Studio Code (VS Code)
- Internet Connection (for package installation and documentation access)
- Quantum Computing Reference Material or Lab Manual

THEORY:

A quantum algorithm is useful when interference raises the probability of the correct answer before the measurement. The four algorithms in this experiment share one pattern: Hadamard gates, one call to an **oracle**, Hadamard gates again, and a measurement. An oracle is a circuit that computes a function f without revealing how it does so. The cost of an algorithm is counted as the number of calls to the oracle, called queries. The standard oracle acts as

$$U_f\,|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$$

Phase kickback. When the last qubit is prepared in the state $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$, the oracle leaves that qubit unchanged and writes the value of $f(x)$ into the sign of the input state:

$$U_f\,|x\rangle|-\rangle = (-1)^{f(x)}\,|x\rangle|-\rangle$$

Deutsch's algorithm. A function f from one bit to one bit is either constant, $f(0) = f(1)$, or balanced, $f(0) \neq f(1)$. A classical method must compute both values, which takes two

queries. The quantum circuit uses one query: the input qubit measures 0 when f is constant and 1 when it is balanced.

Deutsch-Jozsa algorithm. The function now takes n bits and is promised to be either constant or balanced (equal to 0 on exactly half of the inputs). A classical method that must be certain needs $2^{n-1}+1$ queries in the worst case. The quantum circuit needs one: the result is all zeros when f is constant and anything else when it is balanced.

Bernstein-Vazirani algorithm. The function is built from a hidden bit string s:

$$f(x) = s \cdot x = s_1x_1 \oplus s_2x_2 \oplus \cdots \oplus s_nx_n$$

A classical method needs n queries, one for each bit of s. The quantum circuit returns s itself in one query.

Simon's algorithm. The function satisfies $f(x) = f(y)$ exactly when $y = x \oplus s$ for a hidden string s. Each run of the circuit returns a random string z with $z \cdot s = 0 \pmod 2$. About n runs give enough independent equations to solve for s on a classical computer, while a classical search needs on the order of $2^{n/2}$ queries. Simon's algorithm was the first to show an exponential separation in the number of queries, and it led directly to Shor's algorithm.

The Deutsch-Jozsa advantage holds against classical methods that must be certain. A classical method that accepts a small chance of error needs only a few random queries. These algorithms matter because they are the clearest demonstrations of superposition, phase kickback and interference.

Bit ordering in Qiskit. Qiskit prints qubit 0 as the rightmost bit. When a secret string is written into an oracle, place its rightmost character on qubit 0 so that the printed result reads the same as the string.

PROCEDURE:

1. Install Qiskit and the Aer simulator with `pip install qiskit qiskit-aer matplotlib pylatexenc`, and print the installed version with `qiskit._version_`.
2. Open Jupyter Notebook or Visual Studio Code.
3. Import `QuantumCircuit` and `transpile` from `qiskit`, `Statevector` and `Operator` from `qiskit.quantum_info`, `AerSimulator` from `qiskit_aer` and `plot_histogram` from `qiskit.visualization`.
4. Create a circuit with n input qubits, the required output qubits and n classical bits.
5. For Deutsch, Deutsch-Jozsa and Bernstein-Vazirani: apply X and then H to the output qubit to prepare $|-\rangle$, and apply H to every input qubit.
6. Build the oracle from X and CNOT gates and place it between two barriers so that it is

easy to see in the circuit diagram.

7. Apply H to every input qubit again and measure the input qubits only.

8. Draw the circuit with `qc.draw("mpl")` before running it. Add measurements and run the circuit on `AerSimulator` with 1024 shots: `sim.run(transpile(qc, sim), shots=1024).result().get_counts()`. Plot the counts with `plot_histogram`.

9. Interpret the measured bit string according to the algorithm (constant or balanced, the secret string, or an equation for the secret string).

10. Compare the result with the hand calculation. Remember that Qiskit prints qubit 0 as the rightmost bit. Record the code, the circuit diagram, the output and a one-line inference under each program.

PROGRAM:

Program 1: Implement Deutsch's algorithm for all four one-bit functions: $f(x) = 0$, $f(x) = 1$, $f(x) = x$ and $f(x) = \text{NOT } x$. Show the circuit and the measurement for each, and classify each function as constant or balanced.

OUTPUT:

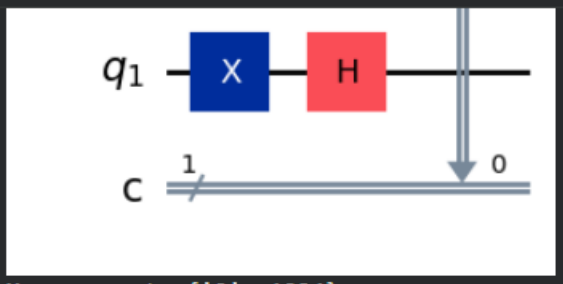
```python
def deutsch_circuit(function):
    qc = QuantumCircuit(2, 1)
    qc.x(1)
    qc.h(0)
    qc.h(1)

    if function == "constant_0":
        pass
    elif function == "constant_1":
        qc.x(1)
    elif function == "identity":
        qc.cx(0, 1)
    elif function == "not":
        qc.x(1)
        qc.cx(0, 1)
    else:
        raise ValueError("Unknown function")

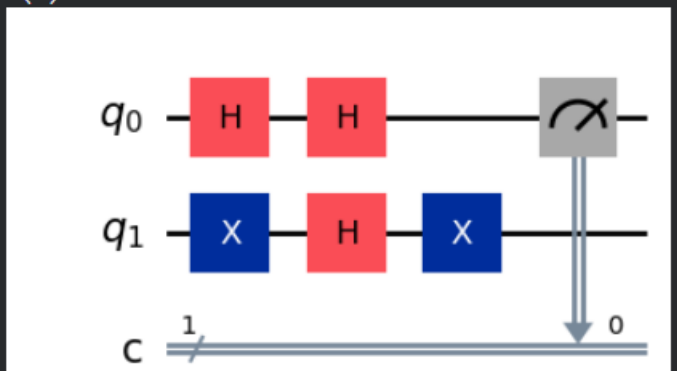
    qc.h(0)
    qc.measure(0, 0)
    return qc

functions = {
    "f(x)=0": "constant_0",
    "f(x)=1": "constant_1",
    "f(x)=x": "identity",
    "f(x)=NOT x": "not",
}
```

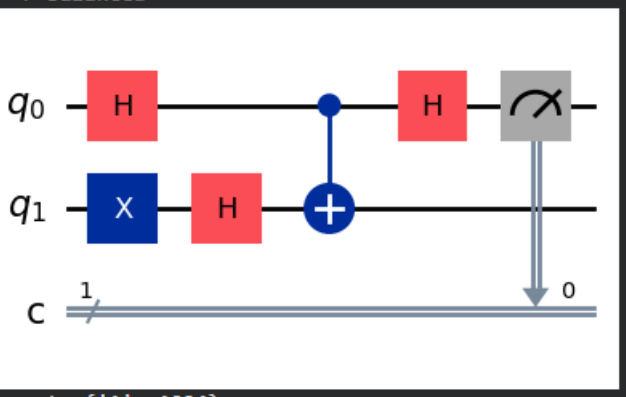
```
for label, kind in functions.items():
    qc = deutsch_circuit(kind)
    print(label, "->", "constant" if kind.startswith("constant") else "balanced")
    display(qc.draw("mpl"))
    counts = sim.run(transpile(qc, sim), shots=SHOTS).result().get_counts()
    print("Measurement:", counts)
```



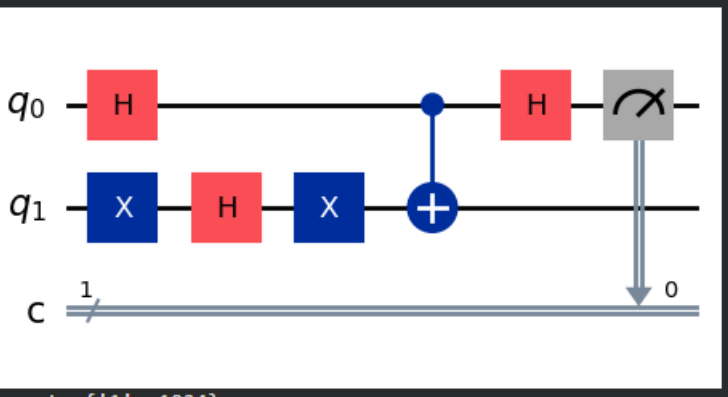

```
Measurement: {'0': 1024}
f(x)=1 -> constant
```



```
Measurement: {'0': 1024}
f(x)=x -> balanced
```




```
Measurement: {'1': 1024}
f(x)=NOT x -> balanced
```



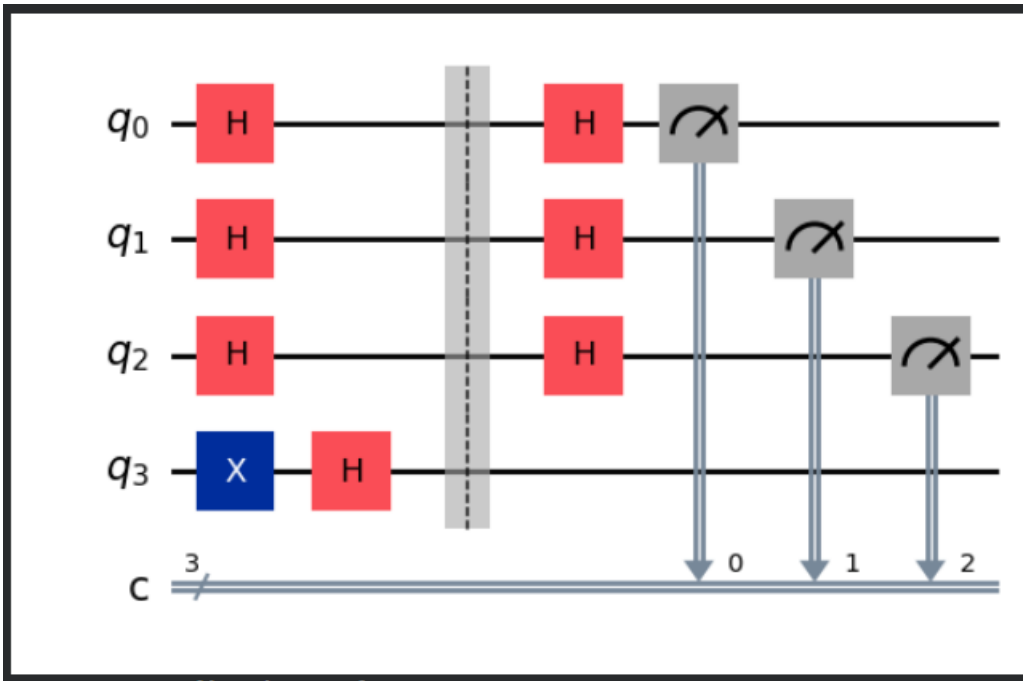

```
Measurement: {'1': 1024}
```

Program 2: Implement the Deutsch-Jozsa algorithm with three input qubits and a constant oracle. Show that the measurement is always 000.

OUTPUT:

```
def dj_constant_circuit(n=3, constant_value=0):
    qc = QuantumCircuit(n + 1, n)
    qc.x(n)
    qc.h(range(n + 1))
    if constant_value == 1:
        qc.x(n)
    qc.barrier()
    qc.h(range(n))
    qc.measure(range(n), range(n))
    return qc

qc = dj_constant_circuit(3, 0)
display(qc.draw("mpl"))
counts = sim.run(transpile(qc, sim), shots=SHOTS).result().get_counts()
print("Measurement:", counts)
```



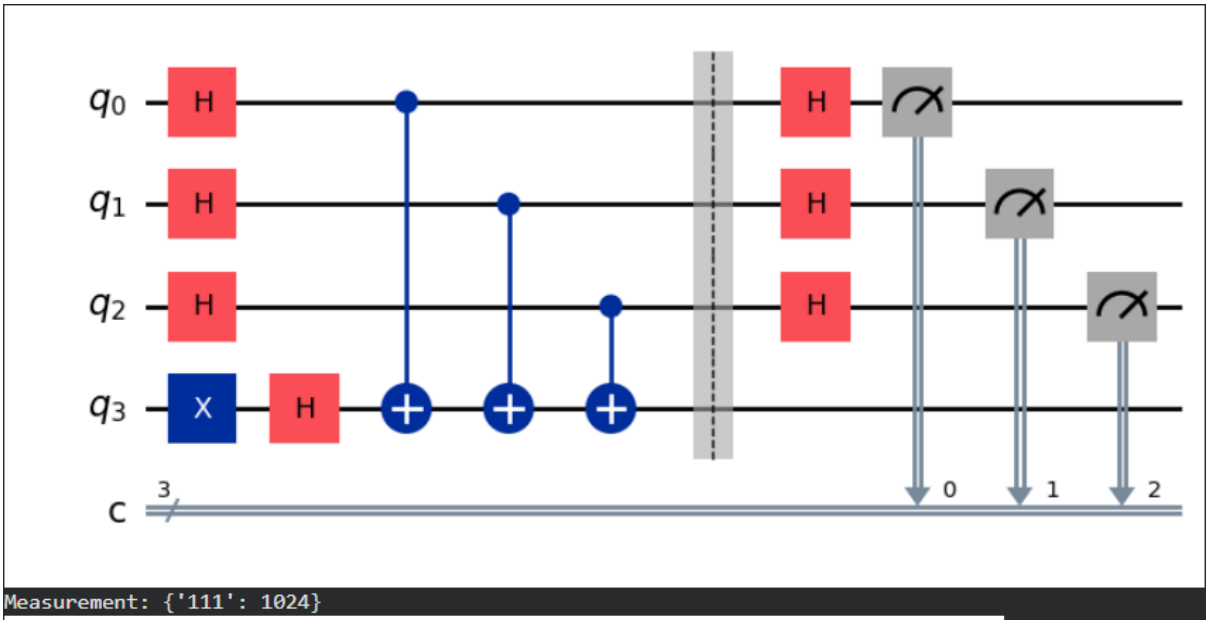

```
Measurement: {'000': 1024}
```

Program 3: Implement the Deutsch-Jozsa algorithm with three input qubits and the balanced oracle $f(x) = x_0 \oplus x_1 \oplus x_2$. Plot the histogram and explain the result.

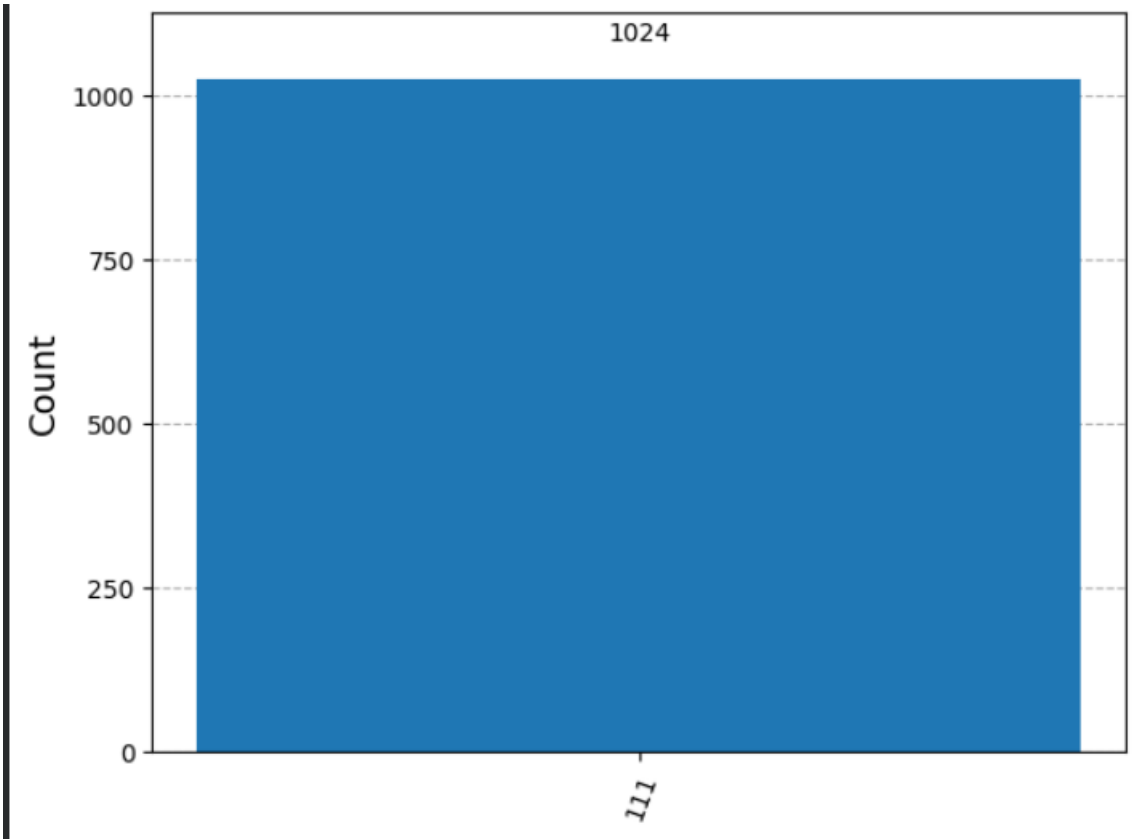
OUTPUT:

```
def dj_parity_circuit(n=3):
    qc = QuantumCircuit(n + 1, n)
    qc.x(n)
    qc.h(range(n + 1))
    for i in range(n):
        qc.cx(i, n)
    qc.barrier()
    qc.h(range(n))
    qc.measure(range(n), range(n))
    return qc

qc = dj_parity_circuit(3)
display(qc.draw("mpl"))
counts = sim.run(transpile(qc, sim), shots=SHOTS).result().get_counts()
print("Measurement:", counts)
display(plot_histogram(counts))
print("Inference: 000 does not appear, so the oracle is balanced.")
```




```
Measurement: {'111': 1024}
```




```
Inference: 000 does not appear, so the oracle is balanced.
```

Program 4: Write a function that returns the Deutsch-Jozsa circuit for any number of input qubits and a chosen oracle type (constant or balanced). Test it for n = 2, 3 and 4.

OUTPUT:

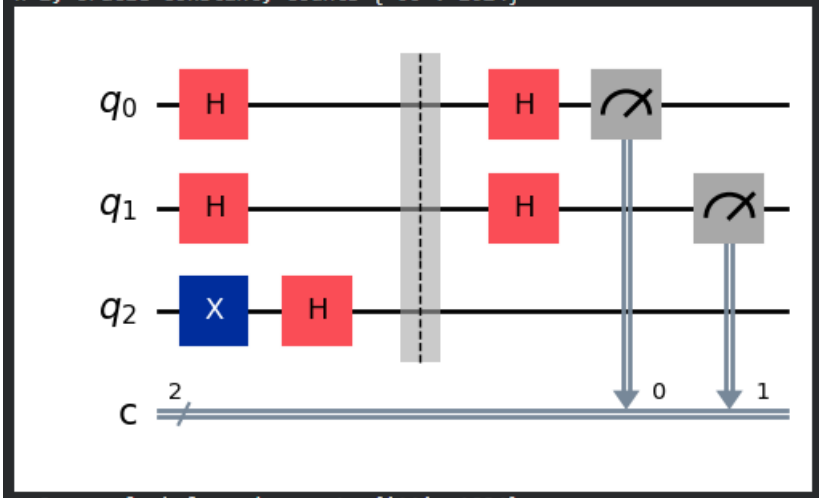
```python
def deutsch_jozsa(n, oracle_type="constant", constant_value=0):
    qc = QuantumCircuit(n + 1, n)
    qc.x(n)
    qc.h(range(n + 1))

    if oracle_type == "constant":
        if constant_value == 1:
            qc.x(n)
    elif oracle_type == "balanced":
        for i in range(n):
            qc.cx(i, n)
    else:
        raise ValueError("oracle_type must be 'constant' or 'balanced'")

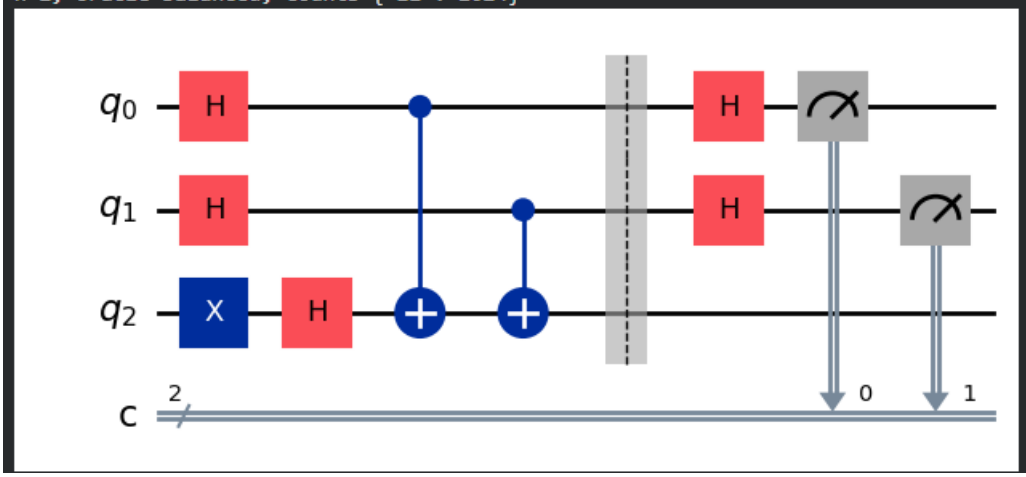
    qc.barrier()
    qc.h(range(n))
    qc.measure(range(n), range(n))
    return qc

for n in [2, 3, 4]:
    for oracle_type in ["constant", "balanced"]:
        qc = deutsch_jozsa(n, oracle_type)
        counts = sim.run(transpile(qc, sim), shots=SHOTS).result().get_counts()
        print(f"n={n}, oracle={oracle_type}, counts={counts}")
        display(qc.draw("mpl"))
```

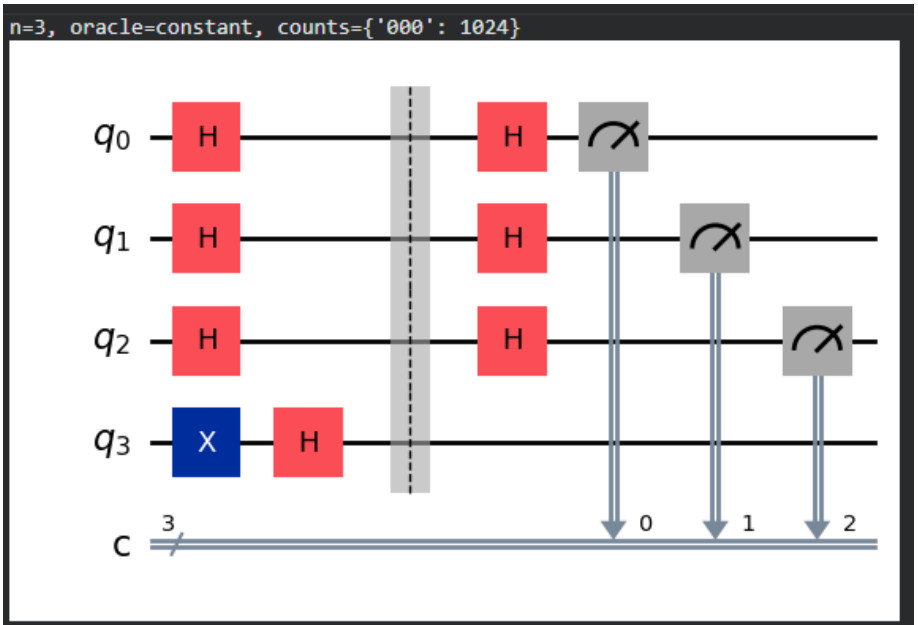
n=2, oracle=constant, counts={'00': 1024}



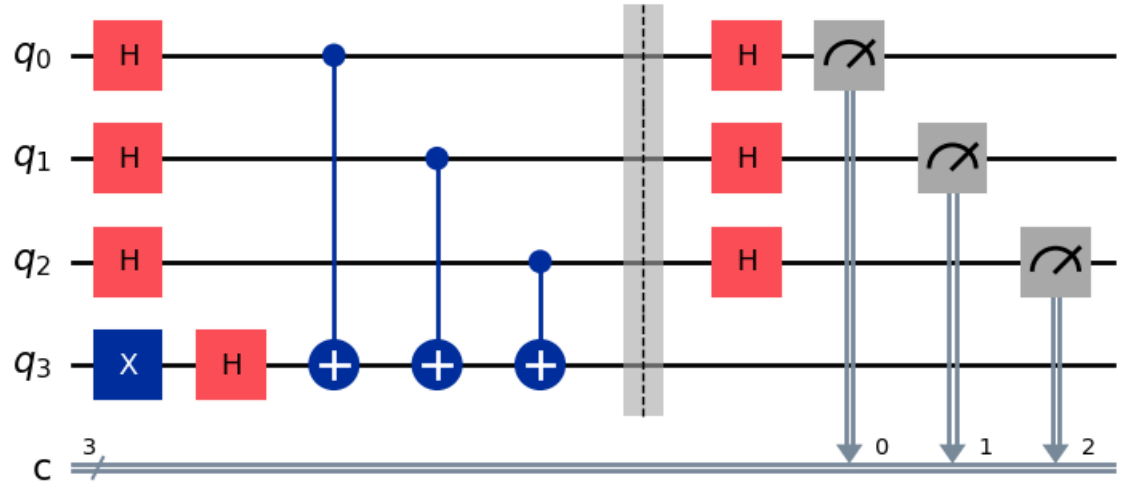
n=2, oracle=balanced, counts={'11': 1024}



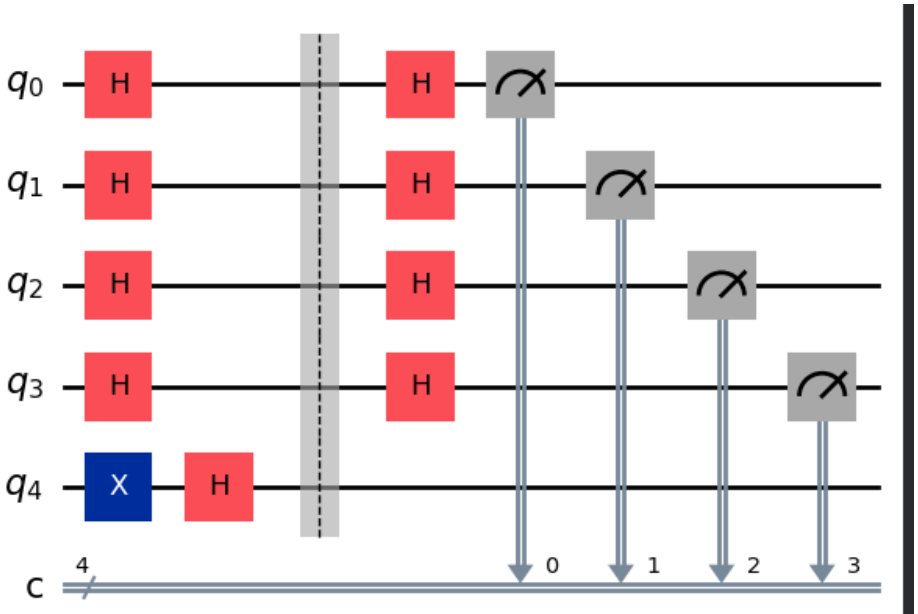
n=3, oracle=constant, counts={'000': 1024}



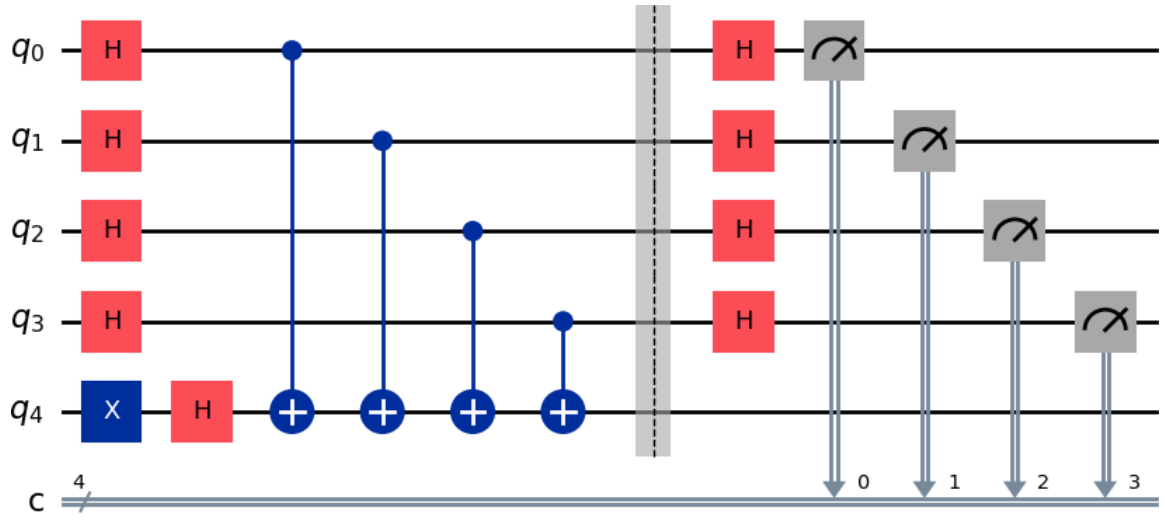
n=3, oracle=balanced, counts={'111': 1024}




```
n=4, oracle=constant, counts={'0000': 1024}
```




```
n=4, oracle=balanced, counts={'1111': 1024}
```

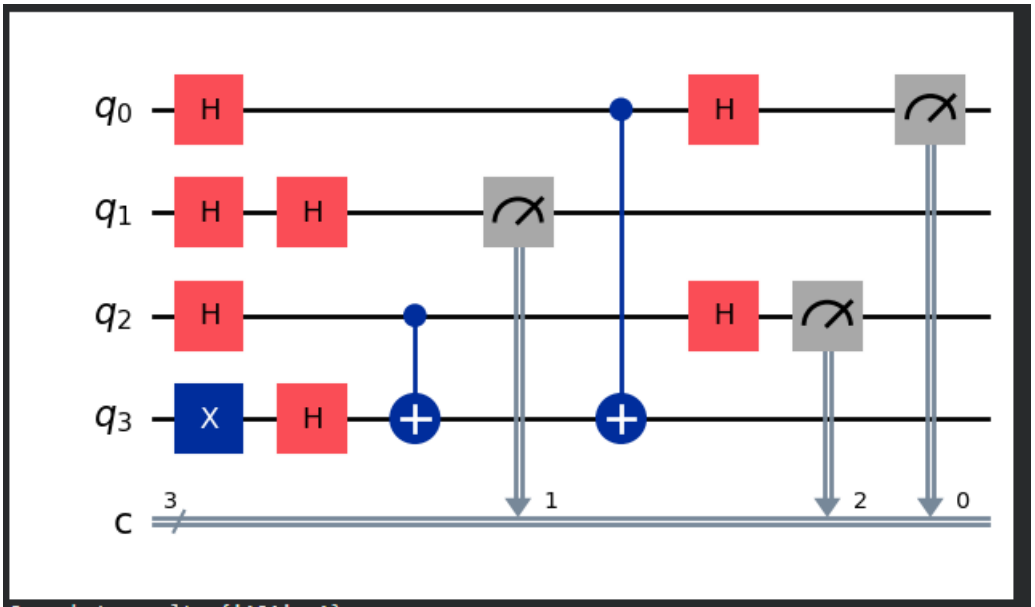


Program 5: Implement the Bernstein-Vazirani algorithm for the secret string s = 101. Verify that a single shot returns the secret string.

OUTPUT:

```
def bv_circuit(secret):
    n = len(secret)
    qc = QuantumCircuit(n + 1, n)
    qc.x(n)
    qc.h(range(n + 1))
    for i, bit in enumerate(secret):
        if bit == "1":
            qc.cx(n - 1 - i, n)
    qc.h(range(n))
    qc.measure(range(n), range(n))
    return qc

secret = "101"
qc = bv_circuit(secret)
display(qc.draw("mpl"))
counts = sim.run(transpile(qc, sim), shots=1).result().get_counts()
print("One-shot result:", counts)
print("Secret string:", secret)
```




```
One-shot result: {'101': 1}
Secret string: 101
```

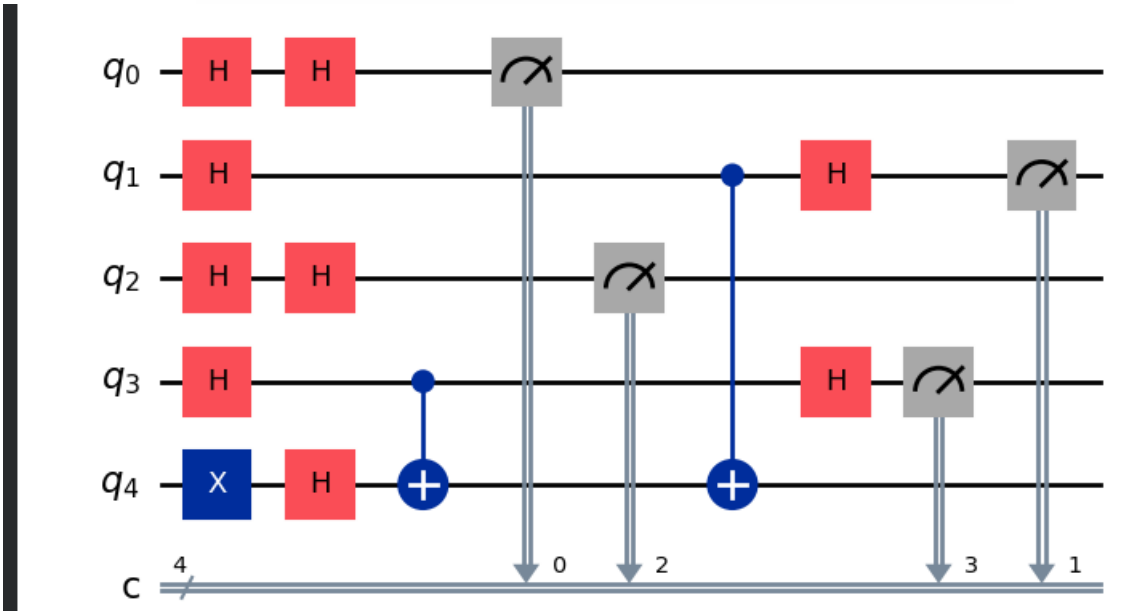
Program 6: Take the last two digits of your register number, find the remainder on division by 16 and write it as a 4-bit string. Use this as the secret string in the Bernstein-Vazirani algorithm and verify the result.

OUTPUT:

```
last_two_digits = 10  # CHANGE this to the last two digits of your register number
secret_number = last_two_digits % 16
secret = format(secret_number, "04b")
print("Remainder modulo 16:", secret_number)
print("Four-bit secret string:", secret)

qc = bv_circuit(secret)
display(qc.draw("mpl"))
counts = sim.run(transpile(qc, sim), shots=1).result().get_counts()
print("One-shot result:", counts)
```

```
Remainder modulo 16: 10
Four-bit secret string: 1010
```



Program 7: Implement Simon's algorithm for n = 2 with the hidden string s = 11. One valid oracle copies the input register to the output register with CNOT gates and then, controlled on one input qubit where s has a 1, applies CNOT gates to the output qubits where s has a 1. Show that every measured string z satisfies $z \cdot s = 0 \pmod 2$.

OUTPUT:

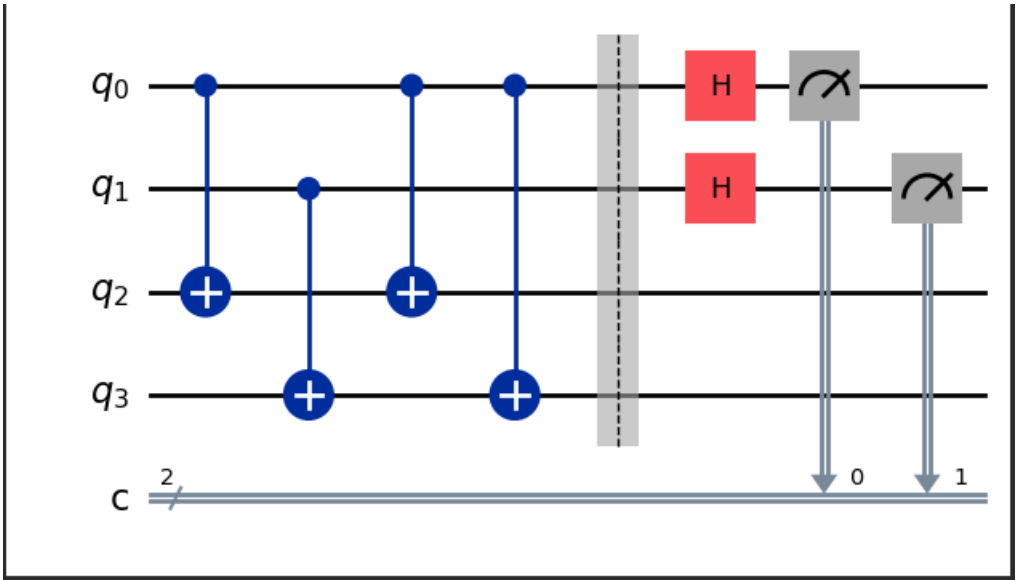
```python
def simon_oracle_circuit(secret):
    n = len(secret)
    qc = QuantumCircuit(2 * n, n)
    # Copy input register to output register.
    for i in range(n):
        qc.cx(i, n + i)

    # Select the first input bit where the secret is 1.
    ones = [i for i, bit in enumerate(secret) if bit == "1"]
    if ones:
        control = ones[0]
        for i, bit in enumerate(secret):
            if bit == "1":
                qc.cx(control, n + i)

    qc.barrier()
    qc.h(range(n))
    qc.measure(range(n), range(n))
    return qc

def parity_dot(a, b):
    return sum(int(x) * int(y) for x, y in zip(a, b)) % 2

secret = "11"
qc = simon_oracle_circuit(secret)
display(qc.draw("mpl"))
counts = sim.run(transpile(qc, sim), shots=SHOTS).result().get_counts()
print("Counts:", counts)
print("Check z dot s = 0:")
for z in counts:
    print(z, parity_dot(z, secret))
```



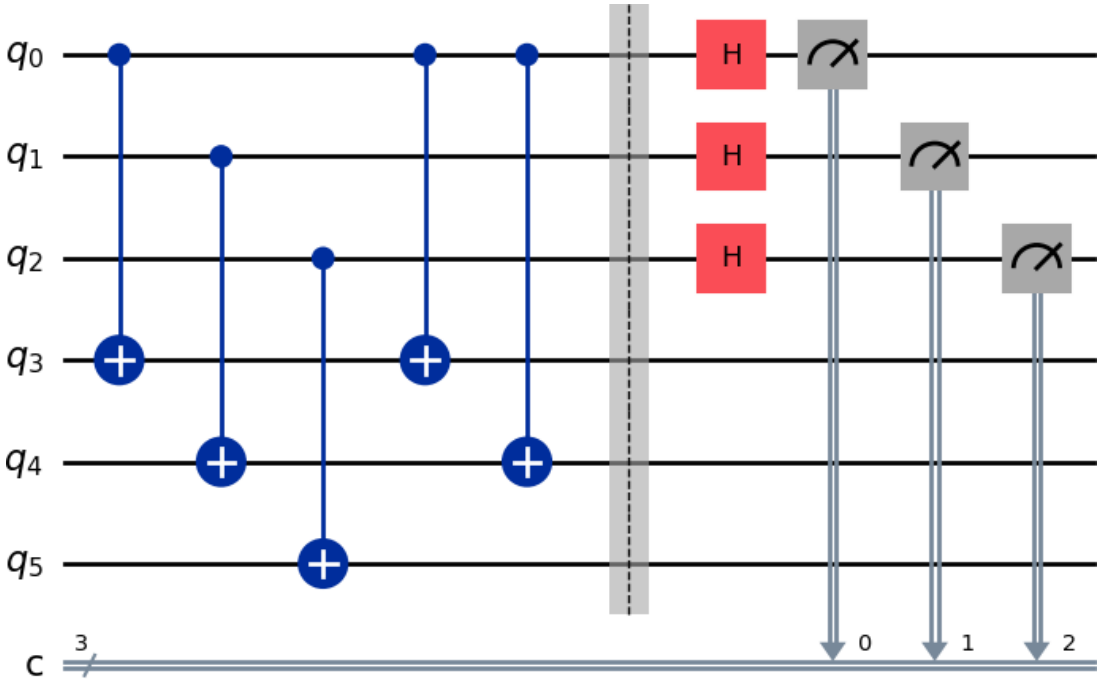

```
Counts: {'01': 235, '11': 270, '10': 247, '00': 272}
Check z dot s = 0:
01 1
11 0
10 1
00 0
```

Program 8: Implement Simon's algorithm for n = 3 with the hidden string s = 110. List the distinct measured strings and solve the equations $z \cdot s = 0$ by hand to recover s.

OUTPUT:

```
secret = "110"
qc = simon_oracle_circuit(secret)
display(qc.draw("mpl"))
counts = sim.run(transpile(qc, sim), shots=SHOTS).result().get_counts()
print("Counts:", counts)
distinct = sorted(counts)
print("Distinct measured strings:", distinct)
print("Dot-product checks:")
for z in distinct:
    print(f"{z} dot {secret} =", parity_dot(z, secret))

print("\nFor s=110, the equation is z0 + z1 = 0 (mod 2), so z0=z1.")
print("Independent equations such as 100 and 010 are not both valid for this secret; use measured strings and solve the modulo-2 system.")
```




```
Counts: {'110': 126, '010': 148, '000': 116, '001': 138, '011': 126, '111': 117, '100': 129, '101': 124}
Distinct measured strings: ['000', '001', '010', '011', '100', '101', '110', '111']
Dot-product checks:
000 dot 110 = 0
001 dot 110 = 0
010 dot 110 = 1
011 dot 110 = 1
100 dot 110 = 1
101 dot 110 = 1
110 dot 110 = 0
111 dot 110 = 0

For s=110, the equation is z0 + z1 = 0 (mod 2), so z0=z1.
Independent equations such as 100 and 010 are not both valid for this secret; use measured strings and solve the modulo-2 system.
```

Program 9: Tabulate the number of oracle queries needed classically and with the quantum algorithm for Deutsch, Deutsch-Jozsa, Bernstein-Vazirani and Simon for n = 3, and comment on the comparison.

OUTPUT:

```
from IPython.display import display, Markdown

table = '''| Algorithm / problem | Classical queries (n=3) | Quantum queries |
|---|---|---:|
| Deutsch (n=1) | 2 for certainty | 1 |
| Deutsch–Jozsa | Up to 5 for certainty (majority threshold is 2^(n-1)+1 = 5) | 1 |
| Bernstein–Vazirani | Up to 3 (one query per bit) | 1 |
| Simon | Exponential in n in the standard black-box setting | O(n) queries to collect enough independent equations |

**Comment:** The quantum algorithms reduce oracle-query complexity in their respective black-box models. Query complexity does not automatically imply a practical speed-up, because real implement
'''
display(Markdown(table))
```

Algorithm / problem	Classical queries (n=3)	Quantum queries
Deutsch (n=1)	2 for certainty	1
Deutsch–Jozsa	Up to 5 for certainty (majority threshold is 2^(n-1)+1 = 5)	1
Bernstein–Vazirani	Up to 3 (one query per bit)	1
Simon	Exponential in n in the standard black-box setting	O(n) queries to collect enough independent equations

Comment: The quantum algorithms reduce oracle-query complexity in their respective black-box models. Query complexity does not automatically imply a practical speed-up, because real implementations also have circuit, noise, and hardware costs.

VIVA Q&A:

1. What is a quantum oracle?

A quantum oracle is a black-box quantum operation that performs a function on quantum states. It is used to provide information about a function in quantum algorithms.

2. What is phase kickback, and which state of the output qubit produces it?

Phase kickback transfers the function value into the phase of the input state. It occurs when the output qubit is prepared in the state

|-> = (|0> - |1>) / sqrt(2)

The function value introduces a phase factor of (-1)^f(x).

3. What is the difference between a constant function and a balanced function?

- Constant function: Gives the same output for every input.
- Balanced function: Gives output 0 for half the inputs and output 1 for the other half.

4. How many queries does a classical method need to solve the Deutsch–Jozsa problem with certainty for n input bits?

In the worst case, a classical deterministic algorithm needs

2^(n-1) + 1

queries to distinguish a constant function from a balanced function with certainty.

5. In the Deutsch–Jozsa algorithm, why does the result 00...0 mean that the function is constant?

The Hadamard gates cause quantum interference. If the function is constant, the amplitudes combine so that the all-zero outcome occurs with probability 1. For a balanced function, that outcome has probability 0.

6. What problem does the Bernstein–Vazirani algorithm solve?

It finds an unknown secret bit string s using an oracle that computes

$f(x) = s \cdot x \bmod 2$

The algorithm determines the secret string.

7. Why can the Bernstein–Vazirani algorithm find the secret string in one query?

The oracle encodes the secret bits into the phases of a superposition. The final Hadamard gates convert those phases into the secret bit string, allowing it to be recovered with one oracle query in the ideal case.

8. State Simon's problem.

Simon's problem is to find a hidden nonzero bit string s for a function satisfying

$f(x) = f(y)$ if and only if $y = x \text{ XOR } s$

The algorithm uses quantum measurements to obtain equations involving s.

9. Why does Simon's algorithm need several runs and classical post-processing?

Each run produces a measurement string z satisfying

$z \cdot s = 0 \bmod 2$

Several independent equations are needed. Classical post-processing solves these equations to recover the hidden string.

10. Is the Deutsch–Jozsa speed-up useful in practice? What happens if a small probability of error is allowed?

Deutsch–Jozsa mainly demonstrates the advantage of quantum computing in oracle-query complexity. Its practical usefulness is limited for many real-world tasks. If a small probability of error is allowed, randomized classical algorithms can often solve the problem with far fewer queries than a deterministic method requiring certainty.

RESULT:

Thus, Deutsch's, Deutsch–Jozsa, Bernstein–Vazirani, and Simon's algorithms were implemented using Qiskit. The constant and balanced functions were classified, secret strings were recovered using quantum oracle circuits, and the oracle-query requirements of classical and quantum approaches were compared.

EVALUATION (for faculty use):

Evaluation method	Prep	Program	Output	Inference	Viva	Timely	Total
Maximum marks	2	5	3	2	2	1	15
Marks awarded							

Faculty signature with date: